vLLM의 LLM 추론 속도 최적화 기법

목차

1. vLLM 소개
2. Attention 최적화(PagedAttention)
3. Batching 최적화(Continuous Batching)
4. 성능 비교 및 추가 기능 소개
5. 코드 동작 구조 및 실 성능 측정
6. 시행착오
7. LLM 프레임워크 비교

1. vLLM(Versatile Large Language Model) 소개

목적

- LLM 모델의 추론 및 서빙 제공을 위한 오픈소스 라이브러리
- 높은 throughput과 낮은 latency를 목표로 설계되어 특히 GPU 환경에서 추론 성능을 극대화

주요 특징

- **PagedAttention**기법과 함께 최적화된 CUDA 커널, Tensor 병렬 처리 등을 활용한 메모리 관리 및 처리 성능 극대화
- **Continuous Batching**을 통한 실시간 배치 처리
- Prefix caching, KV cache Quantization 등 여러 추론 최적화 추가 기능 제공

2. Attention 최적화

2.1. Attention 연산에서 KV Cache

2.2. KV Cache 동작 메커니즘

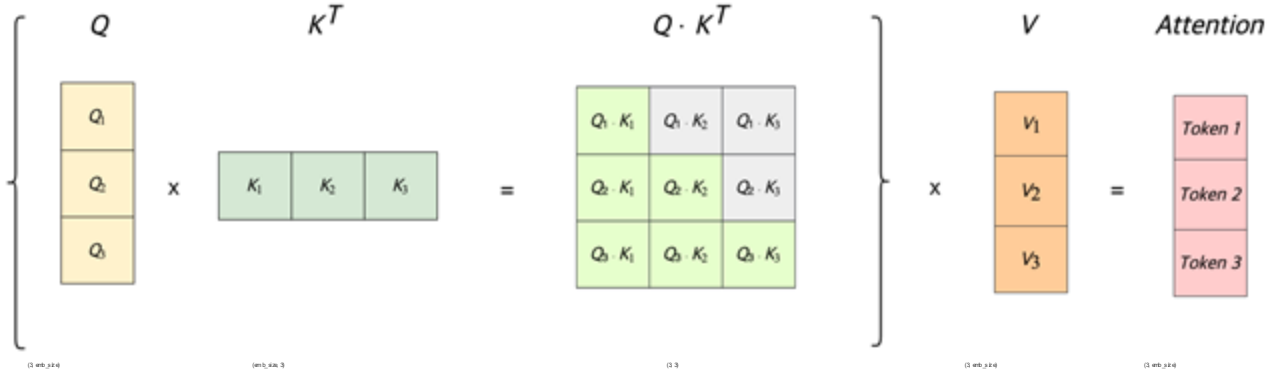
2.3. KV Cache의 문제점

2.4. vLLM PagedAttention

2.1. Attention 연산에서 KV Cache

Attention 메커니즘 특징

- attention을 통해 각 토큰 간의 연관성을 계산하여 각 토큰이 어떤 정보를 얼마나 참고할지 결정
- attention 과정에서 이전 토큰들에 대한 벡터 연산이 수행됨

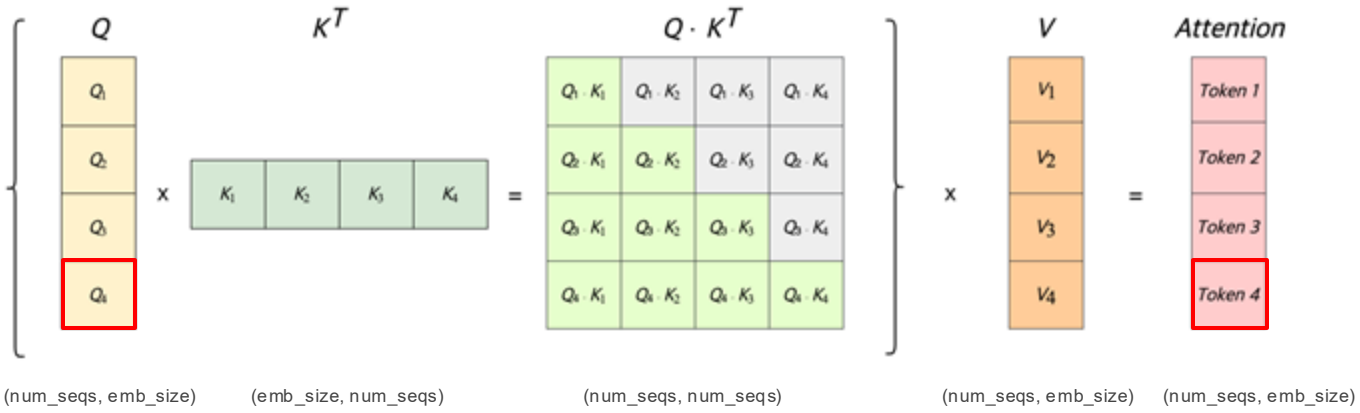


$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

2.1. Attention 연산에서 KV Cache

Attention 메커니즘 특징

- 새로운 Query에 대하여 Attention 계산 시, 이전에 계산했던 과정을 동일하게 진행해야 되는 비효율 발생

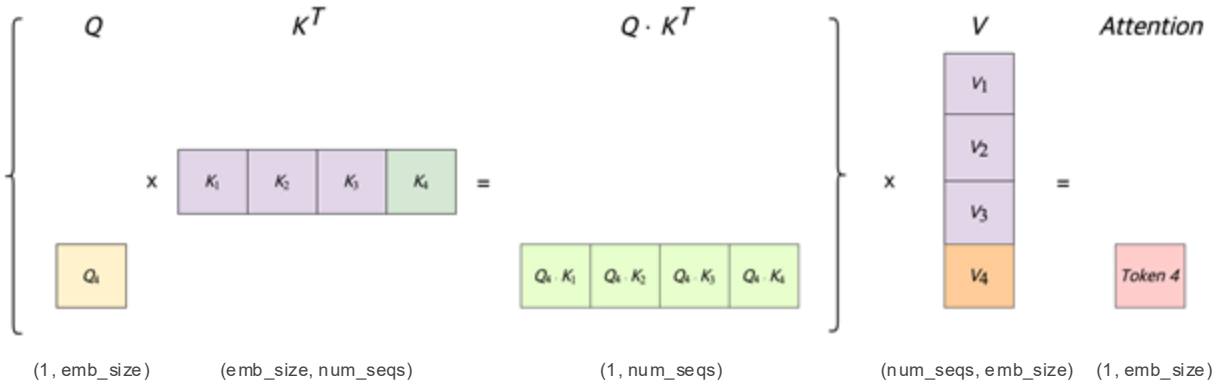


- Attention 연산 시 Attention Score 계산을 위한 QK^T 연산이 가장 큰 병목
- 1개의 토큰에 대해서 Decode 시 시간 복잡도: $\mathcal{O}(n^2 \cdot d)$ n = num_seqs: 전체 입력 토큰 수
  d = emb_size: hidden state 차원 크기

2.1. Attention 연산에서 KV Cache

KV Cache: 반복 연산에 사용되는 K, V 벡터에 대해 메모리에 Cache를 해두고 재사용

- n번째 토큰 예측: 1~ (n-1) 개의 K, V 벡터만을 통해 마지막 hidden state, h_n 을 구할 수 있음



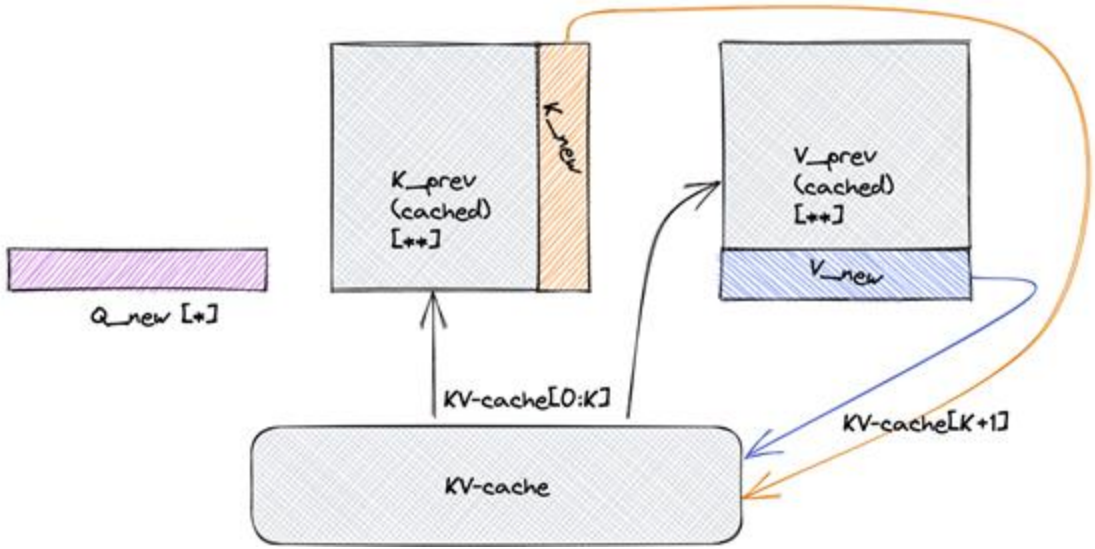
- 1개의 토큰에 대해서 Decode 시 시간 복잡도: $\mathcal{O}(n \cdot d)$

  n = num_seqs: 전체 입력 토큰 수
  d = emb_size: hidden state 차원 크기

- KV Cache를 통해 시간복잡도를 크게 줄일 수 있음, KV Cache 도입 후 LLM 추론은 **Memory-Bound** 작업으로 바뀜

2.2. KV Cache 동작 메커니즘

KV Cache를 통한 Attention



1. 새로운 Query N에 대해서 Attention 연산을 위해 이전에 계산된 Key, Value Vector를 메모리에서 Load
2. 새롭게 연산된 N번째 K, V 값을 KV Cache 메모리에 저장

2.3. KV Cache 문제점

높은 KV Cache 메모리 사용량

KV Cache Memory Usage: batch_size * n_layers * hidden_size * seq_len * 2(K&V) * 2Bytes(float16)

(Key, Value Vector 크기)

Qwen-14B 모델 기준 메모리 사용량 계산

Qwen-14B Config

n_layers	40
hidden_size	5120
seq_len	4096

메모리 사용량

- Model Size : **26GiB**
- KV Cache: batch_size * 40 * 5120 * 4096 * 2 * 2B = **3.12GiB** * batch_size

Batch Size를 **10**으로 설정 시, 모델 크기보다 큰 **31.2GiB**가 사용됨

2.3. KV Cache 문제점

메모리 I/O 증가

HuggingFace Transformers KV Cache 연산

Dynamic Cache

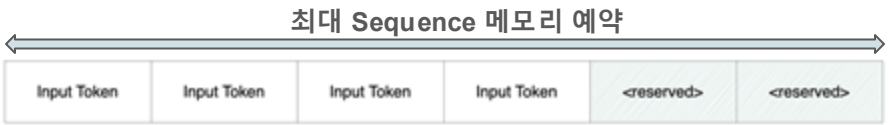
- Input Token 마다 KV Cache 메모리 할당량이 달라짐

```
def forward(

    query_states = query_states.view(shape_q).transpose(1, 2)
    key_states = key_states.view(shape_kv).transpose(1, 2)
    value_states = value_states.view(shape_kv).transpose(1, 2)

    if layer_past is not None:
        past_key, past_value = layer_past
        key_states = torch.cat((past_key, key_states), dim=-2)
        value_states = torch.cat((past_value, value_states), dim=-2)
```

- 새로운 K, V 값을 Vector에 추가할 때마다 concat 연산을 통해 병합
- torch.cat은 메모리 copy 및 reallocation을 수행하여 메모리 연산 병목이 생김

Static Cache

- 최대 Sequence를 미리 메모리에 할당함



- concat 연산을 하지 않으므로 메모리 copy 및 reallocation을 수행할 필요가 없음
- 항상 최대 메모리를 미리 할당하기 때문에 내부 단편화 문제 발생

2.3. KV Cache 문제점

메모리 단편화 문제

- KV Cache 특성상 내부, 외부 메모리 단편화가 발생함

내부 메모리 단편화

최대 Sequence 메모리 예약

저	는	LLM	엔지니어	입니다	<reserved>	<reserved>	<reserved>	<reserved>	<reserved>
<PAD>	좋은	발표를	하고	싶어요	<reserved>	<reserved>	<reserved>	<reserved>	<reserved>

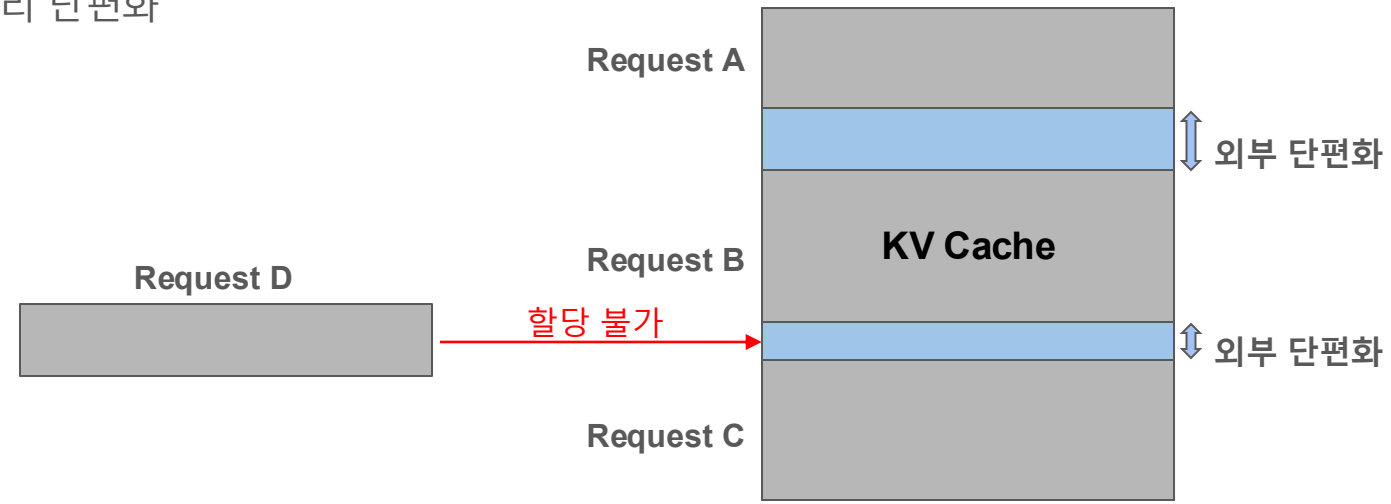
사용하지 않지만 미리 예약된 메모리로 내부 단편화

- KV Cache를 위해 요청 별 max sequence 만큼 예약함
- 항상 max sequence만큼 사용하지 않기 때문에 메모리 단편화 낭비 발생

2.3. KV Cache 문제점

메모리 단편화 문제

외부 메모리 단편화



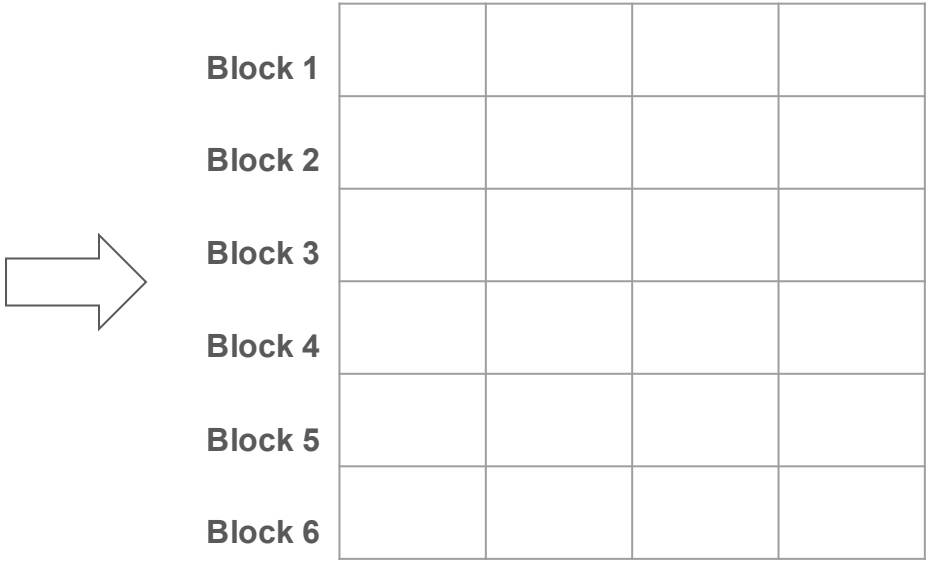
- 할당된 요청별 메모리 사이사이에 누수되는 공간 발생
- 실제 GPU VRAM을 온전히 활용하는 것이 불가능함

2.4. vLLM PagedAttention

vLLM은 KV Cache 메모리를 **Block** 단위로 분리하여 관리하는 **PagedAttention** 기법 사용

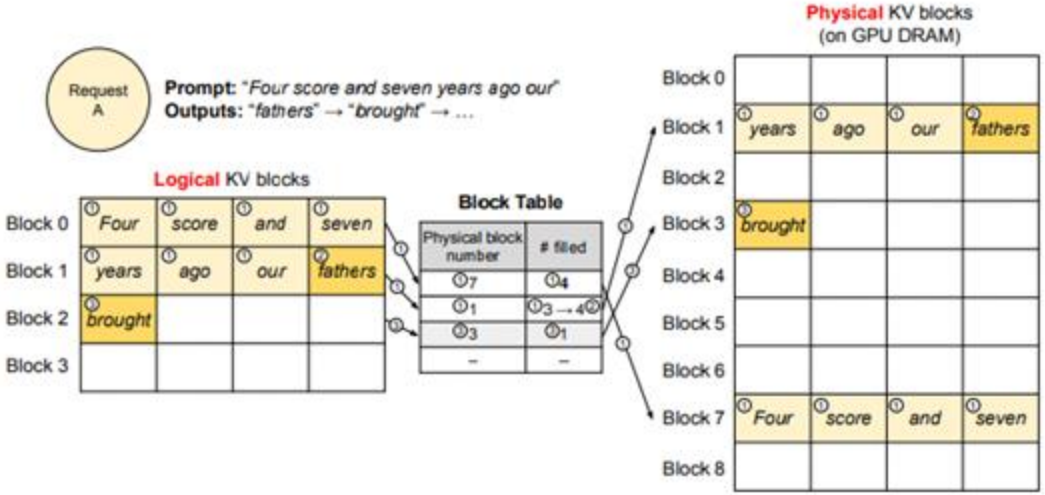
고정된 단일 물리적 공간

Block 단위로 분리된 공간



2.4. vLLM PagedAttention

PagedAttention 동작 원리

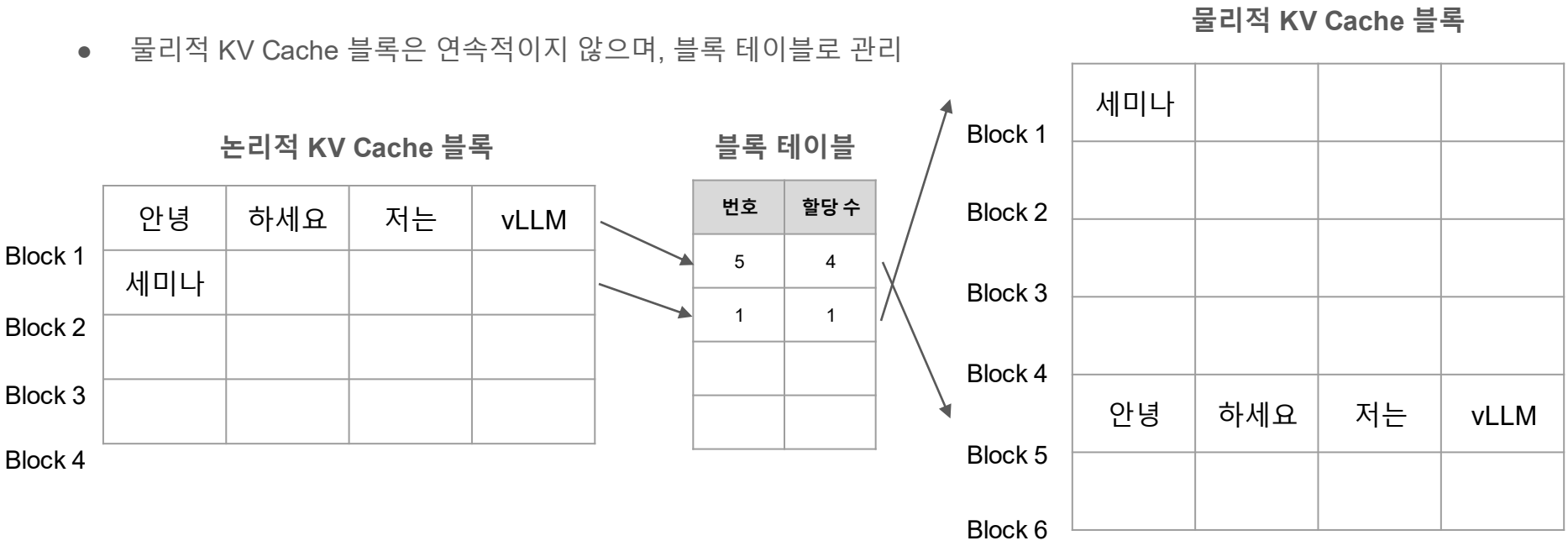


- **논리적**, **물리적** 메모리 공간을 분류하여 Block 단위로 메모리 관리하고, **Block 테이블**로 논리적 메모리 공간과 물리적 메모리 공간을 연결
- 하나의 Block에 채울 공간이 없다면 그다음 토큰들은 새로운 Block에 할당함
- 논리적 공간에 하나의 요청에 대한 토큰들을 연속적으로 관리하지만 실제 물리적 메모리 공간에는 **비연속적**으로 Block 단위로만 저장됨

2.4. vLLM PagedAttention

PagedAttention 동작 예시

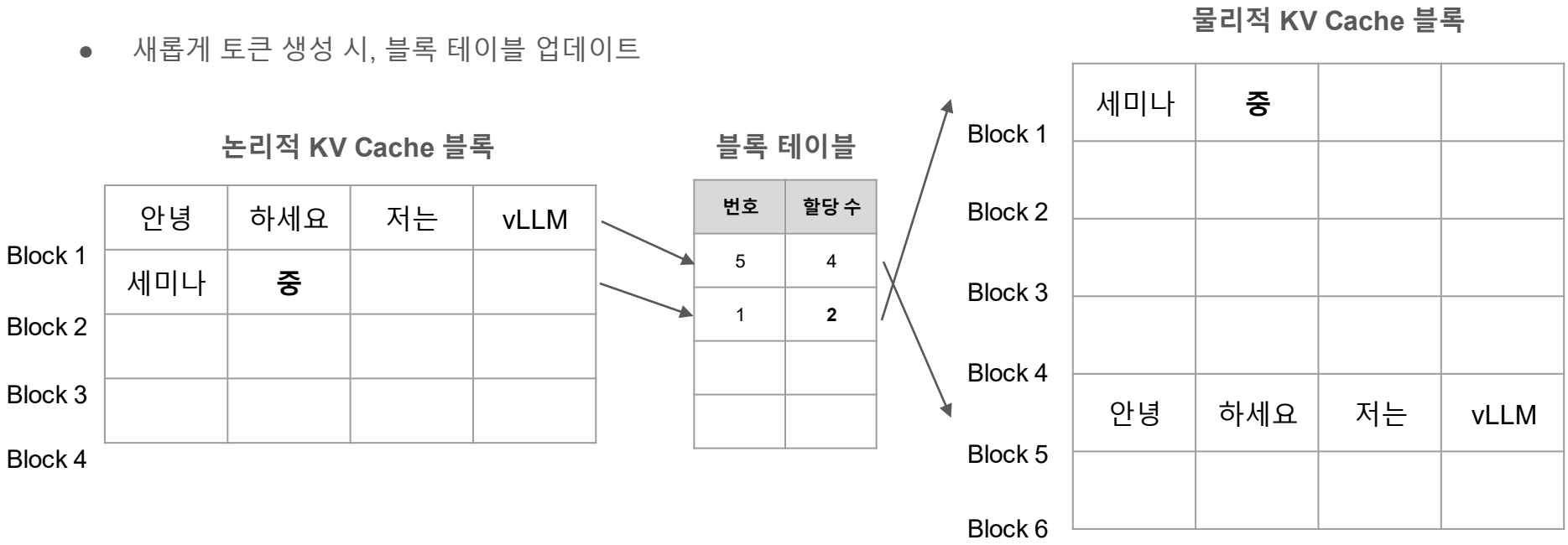
- 물리적 KV Cache 블록은 연속적이지 않으며, 블록 테이블로 관리



2.4. vLLM PagedAttention

PagedAttention 동작 예시

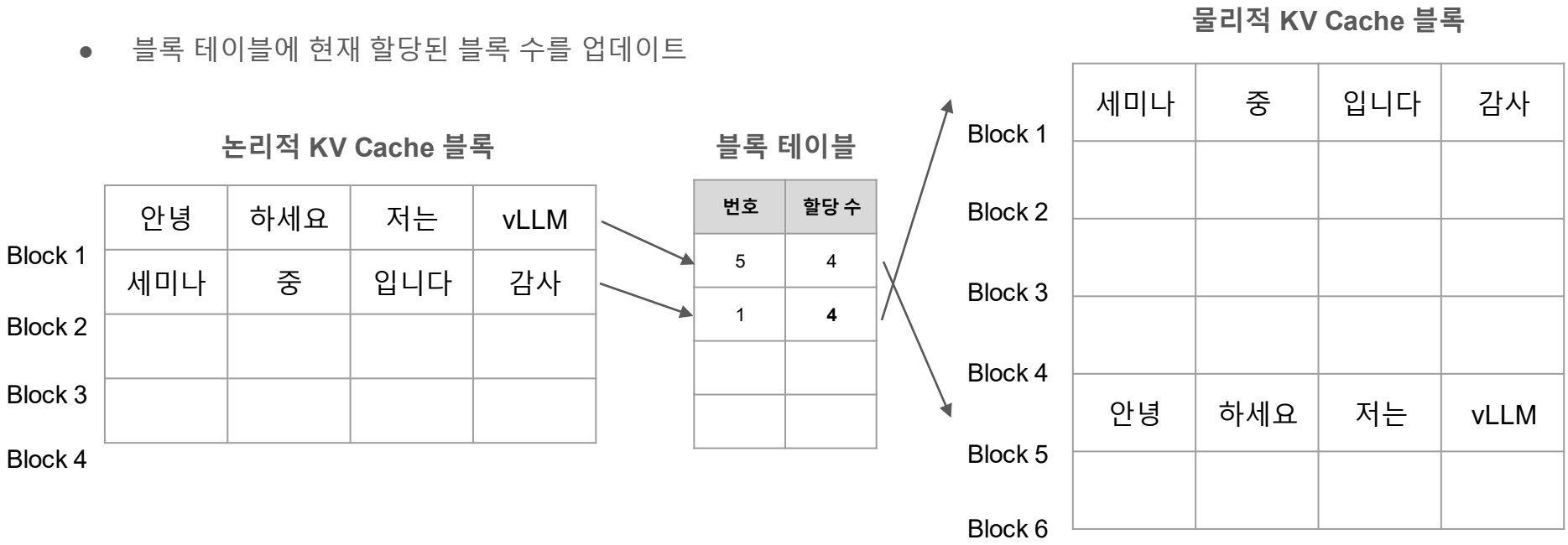
- 새롭게 토큰 생성 시, 블록 테이블 업데이트



2.4. vLLM PagedAttention

PagedAttention 동작 예시

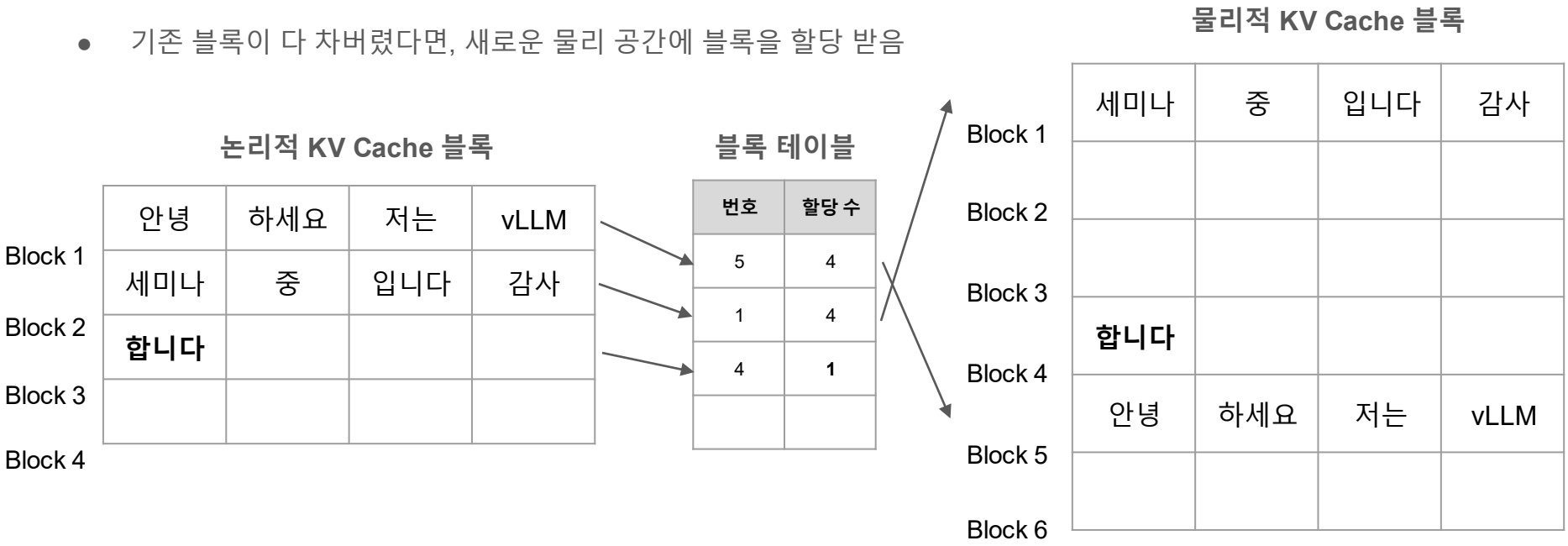
- 블록 테이블에 현재 할당된 블록 수를 업데이트



2.4. vLLM PagedAttention

PagedAttention 동작 예시

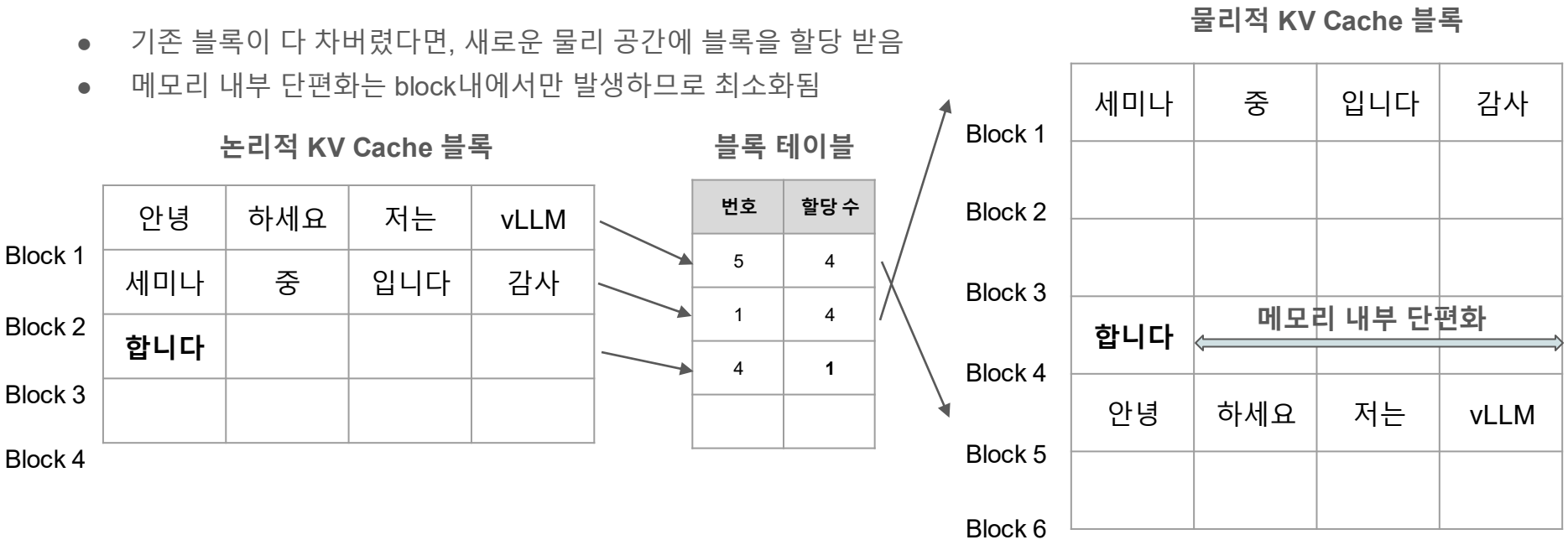
- 기존 블록이 다 차버렸다면, 새로운 물리 공간에 블록을 할당 받음



2.4. vLLM PagedAttention

PagedAttention 동작 예시

- 기존 블록이 다 차버렸다면, 새로운 물리 공간에 블록을 할당 받음
- 메모리 내부 단편화는 block내에서만 발생하므로 최소화됨



2.4. vLLM PagedAttention

Block별 Key, Value 벡터 분산 계산

$$K_j = (k_{(j-1)B+1}, \ldots, k_{jB})$$

$$V_j = (v_{(j-1)B+1}, \ldots, v_{jB})$$

Block size B = 4 일 때, j=[1,2] 라면

- $K_1 = (k_1, k_2, k_3, k_4)$

  → j = 1 → (1−1)*4+1 = 1 ~ 4

- $K_2 = (k_5, k_6, k_7, k_8)$

  → j = 2 → (2−1)*4+1 = 5 ~ 8

- $[k_1, k_2, k_3, k_4, k_5, k_6, k_7, k_8]$
  (총 8개의 Key 토큰)

2.4. vLLM PagedAttention

Block별 Attention Score, Output 분산 계산

$$A_{ij} = \frac{\exp(q_i^\top K_j/\sqrt{d})}{\sum_{t=1}^{\lceil i/B \rceil} \exp(q_i^\top K_t \mathbf{1}/\sqrt{d})}, \quad o_i = \sum_{j=1}^{\lceil i/B \rceil} V_j A_{ij}^\top,$$

$$A_{ij} = (a_{i,(j-1)B+1}, \ldots, a_{i,jB})$$

- j번째 block, i번째 Query에 대한 Attention Score, Output 벡터 계산
- 오른쪽 예시의 경우 **j = [1,2,3], B = 4, i = 9** 을 가짐

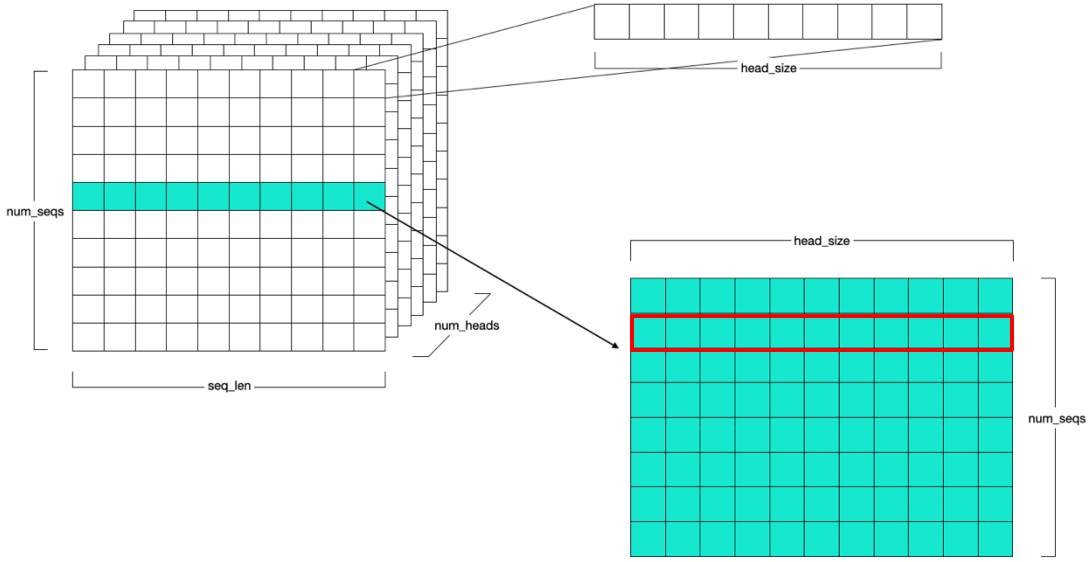
물리적 KV Cache 블록

B=4

j=2	Block 1	세미나	중	입니다	감사
	Block 2				
	Block 3				
		i=9 합니다			
j=3	Block 4	안녕	하세요	저는	vLLM
j=1	Block 5				
	Block 6				

2.4. vLLM PagedAttention

Cuda 연산 최적화



각각의 쿠다 블럭은 색칠해진 영역을 담당

- Grid 크기: [num_heads, num_sequences] 크기의 2차원
- Cuda 블록 크기: [128] 크기의 1차원 (NVIDIA A100 기준)
- 각 Cuda 블록은 하나의 Sequence와 하나의 헤드에 대한 연산을 수행하도록 병렬화
- 각 Cuda 블록은 총 128개의 Thread에서 나누어 병렬적으로 계산하고 최종적으로 Reduce하여 연산

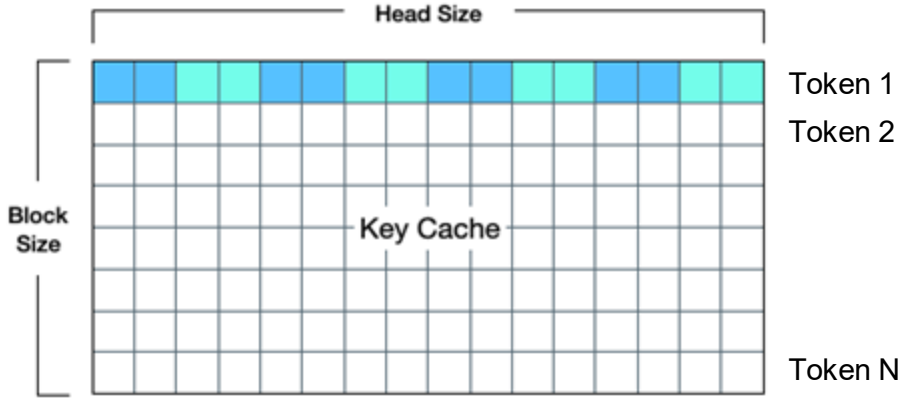
* 각 Cuda 블록이 처리하는 벡터

$Q \in R^{1\times h}$ L :Sequence 길이

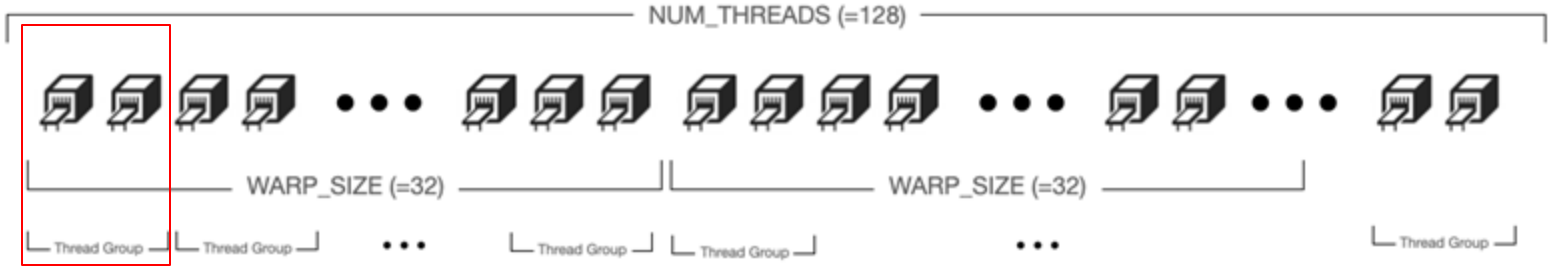
$K, V \in R^{L\times h}$ h :head size

2.4. vLLM PagedAttention

Cuda 연산 최적화

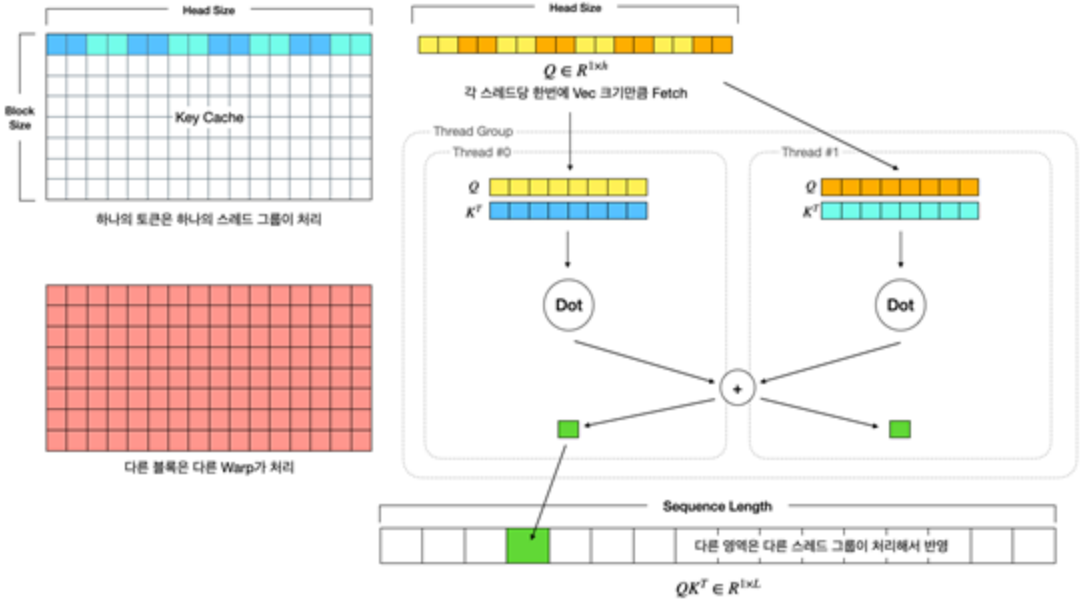


- 하나의 블록은 하나의 **WARP이** 담당함
- WARP은 **Thread Group**들로 구성되어 있음
- 하나의 Thread Group은 하나의 Token을 담당하여 계산, 2개의 Thread로 구성되어 있음
- **2개의 Thread**가 하나의 Token을 계산하는 구조



2.4. vLLM PagedAttention

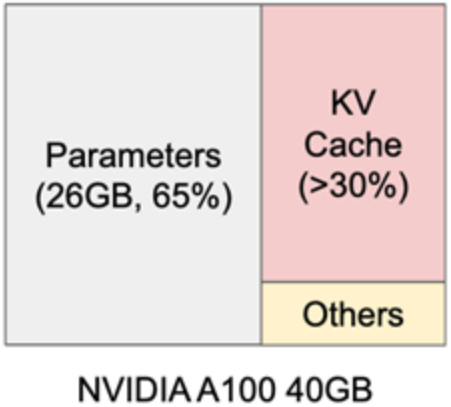
Cuda 연산 최적화

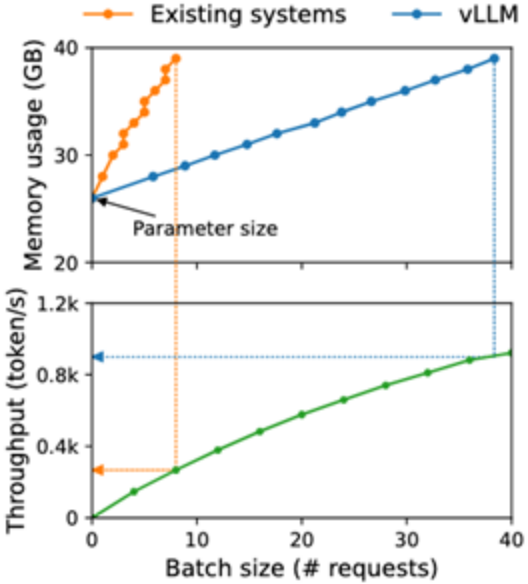


- Thread Group내의 각 Thread는 계산된 Key, Query 값을 Fetch
- 각 Thread는 Fetch된 Q, K값으로 QK 값 계산
- 각 계산된 결과를 합하여 최종 출력값을 저장

2.4. vLLM PagedAttention

PagedAttention을 통한 이점

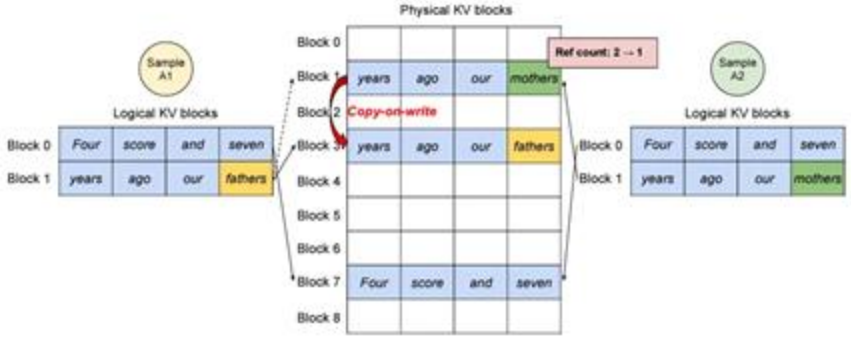




- 평균적으로 KV Cache에 할당되는 메모리는 **30%** 이상임
- 기존 시스템은 비효율적인 KV Cache 메모리 관리에 따라 최대 10 배치까지 밖에 수행하지 못했음
- PagedAttention을 적용한 vLLM은 효율적인 KV Cache 관리로 40배치 이상 수행이 가능했음

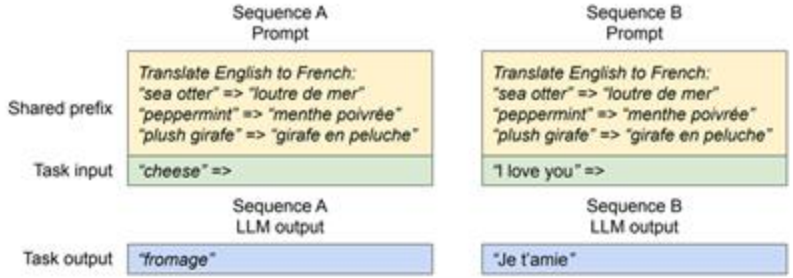
2.4. vLLM PagedAttention

Parallel Sampling



- 일반적으로 동일한 프롬프트에 대해 여러개의 시퀀스를 생성하는 Sampling 기법에서도 메모리 효율성이 높음
- Block단위의 Copy-on-write가 가능, 중복되는 부분은 동일 Block으로 참조하고, 다른 부분만 Copy하여 사용

Shared Prefix



- Parallel Sampling과 마찬가지로 동일한 System 프롬프트에 대해서는 KV값이 동일하기 때문에, KV Cache를 공유하여 사용함
- 이 경우에도 마찬가지로 동일한 KV Block 공유를 통해 메모리 효율성을 높힐 수 있음

3. Batching 최적화

3.1. Dynamic Batching

3.2. Continuous Batching

3.1. Dynamic Batching

- 가장 길이가 긴 입력값과 행렬곱을 위해 길이를 맞추는 Padding 과정 필요

Request A	Input 1	Input 2	Input 3	Input 4			
Request B	<PAD>	Input 1	Input 2	Input 3			
Request C	<PAD>	<PAD>	<PAD>	Input 1			

3.1. Dynamic Batching

- 배치 단위로 Output 토큰 동시 생성

Request A	Input 1	Input 2	Input 3	Input 4	Output 1		
Request B	<PAD>	Input 1	Input 2	Input 3	Output 1		
Request C	<PAD>	<PAD>	<PAD>	Input 1	Output 1		

3.1. Dynamic Batching

- 이미 처리가 끝난 요청은 다른 요청이 끝날 때 까지 대기해야함 (Request B)

Request A	Input 1	Input 2	Input 3	Input 4	Output 1	Output 2	
Request B	<PAD>	Input 1	Input 2	Input 3	Output 1	<EOS>	
Request C	<PAD>	<PAD>	<PAD>	Input 1	Output 1	Output 2	

3.1. Dynamic Batching

- 이미 처리가 끝난 요청은 다른 요청이 끝날 때 까지 대기하며 지속적으로 <EOS> 토큰을 추가함

Request A	Input 1	Input 2	Input 3	Input 4	Output 1	Output 2	<EOS>
Request B	<PAD>	Input 1	Input 2	Input 3	Output 1	<EOS>	<EOS>
Request C	<PAD>	<PAD>	<PAD>	Input 1	Output 1	Output 2	Output 3

- 배치 진행 시 새로운 요청은 배치 작업이 끝날 때 까지 대기해야함

모든 요청 종료시 까지 처리 불가

Request D

3.1. Dynamic Batching

Dynamic Batching의 단점

- Padding 처리와 같은 비효율적인 배치 처리 및 모든 요청이 끝날 때 까지 먼저 끝난 요청은 내뱉을 수 없음
- 배치 처리 중 새로운 요청이 들어와도 처리가 끝날 때 까지 수행되지 못하고 대기해야 함

3.2. Continuous Batching

vLLM은 Dynamic Batching의 한계를 극복한 Continuous Batching 기법을 제공

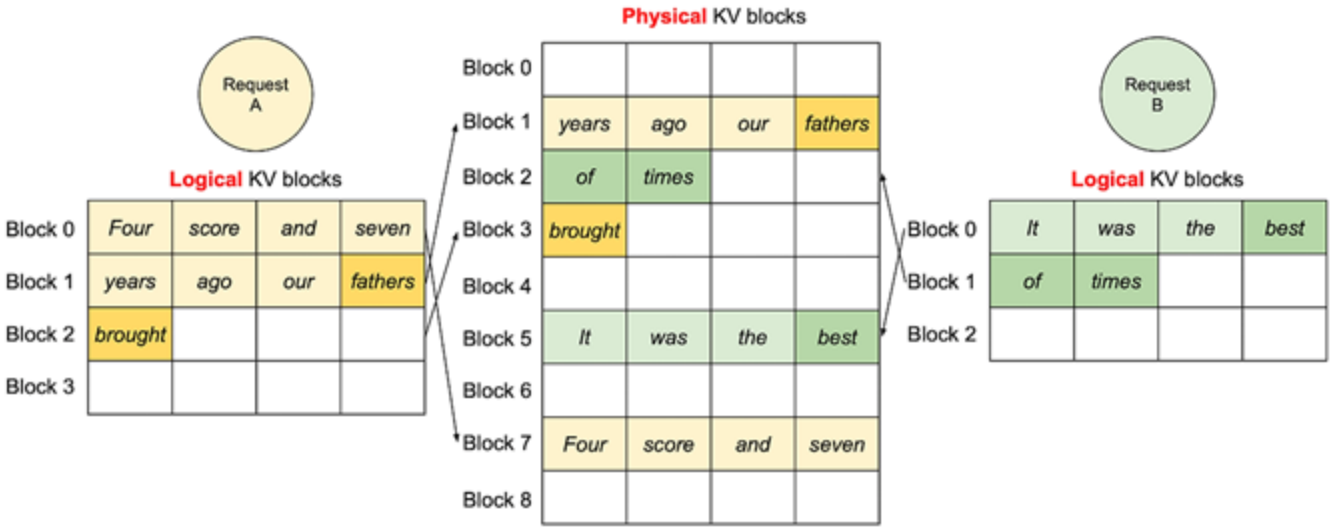
T1	T2	T3	T4	T5	T6	T7	T8
S1	S1	S1	S1				
S2	S2	S2					
S3	S3	S3	S3				
S4	S4	S4	S4	S4			

T1	T2	T3	T4	T5	T6	T7	T8
S1	S1	S1	S1	S1	End	S6	S6
S2	S2	S2	S2	S2	S2	S2	End
S3	S3	S3	S3	End	S5	S5	S5
S4	S4	S4	S4	S4	S4	End	S7

- 요청이 끝난 배치에 대해서 즉시 반환되며, max batch 크기 까지의 새로운 요청을 받을 수 있음
- vLLM은 padding 처리를 하지 않아도 되는 Continuous Batching을 설계함

3.2. Continuous Batching

논리적, 물리적 KV Cache 블록에서 동시 처리



- 각 요청은 논리적 KV Blocks으로 구분되어 관리되며, 비연속적인 물리적 공간에 각 요청별 블록을 할당받음

3.2. Continuous Batching

vLLM Continuous Batching의 텐서 처리

- vLLM Continuous Batching은 2차원의 배치 배열을 1차원으로 Flatten 처리함
- 불필요한 Padding 처리가 필요 없음, 메모리 효율성 증가

3.2. Continuous Batching

Batch 처리 시, Flatten 처리를 통해 2차원 배열을 1차원 배열로 reshape

- Attention 연산 전, LayerNorm 및 Linear 처리를 1차원 배열로 한번에 처리

Block 1	세미나	중		
Block 2				
Block 3	저는	LLM		
Block 4				
Block 5	안녕	하세요	저는	vLLM
Block 6				

Request A: [안녕, 하세요, 저는 ,vLLM, 세미나, 중]

Request B: [저는, LLM]

↓ **PagedAttention 함수 전달 시 Flatten**

Input Tokens: [안녕, 하세요, 저는 ,vLLM, 세미나, 중, 저는, LLM]

Input Positions: [0, 1, 2, 3, 4, 5, 0, 1]

Slot Mapping: [20, 21, 22, 23, 0, 1, 8, 9]

↓ **Attention 연산 시 reshape**

Input Tokens[:RequestA] -> [안녕, 하세요, 저는 ,vLLM, 세미나, 중]

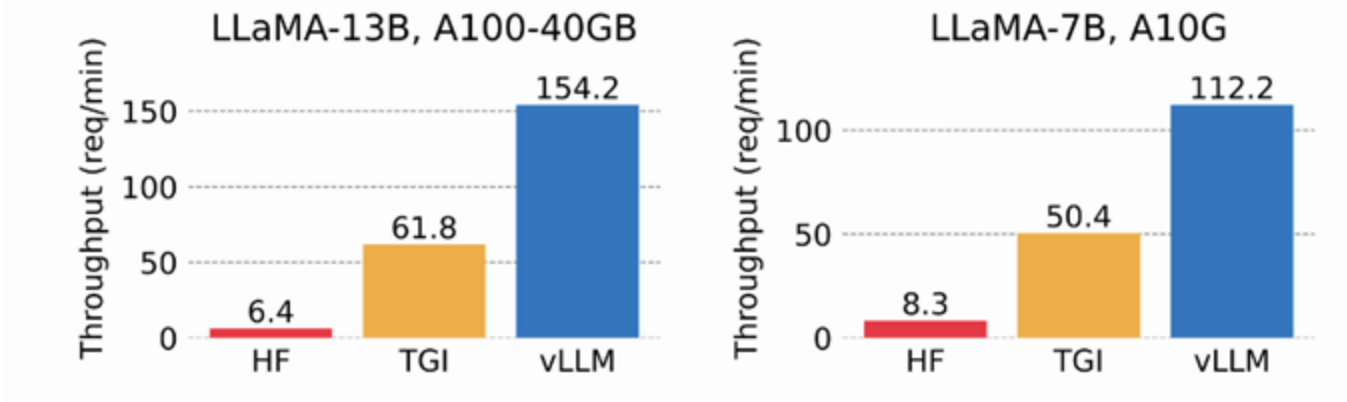
Input Tokens[:RequestB] -> [저는, LLM]

4. 성능 비교 및 추가 기능 소개

4.1. 성능 비교

4.2. 추가 기능 소개: Speculative Decoding

4.1. 성능 비교

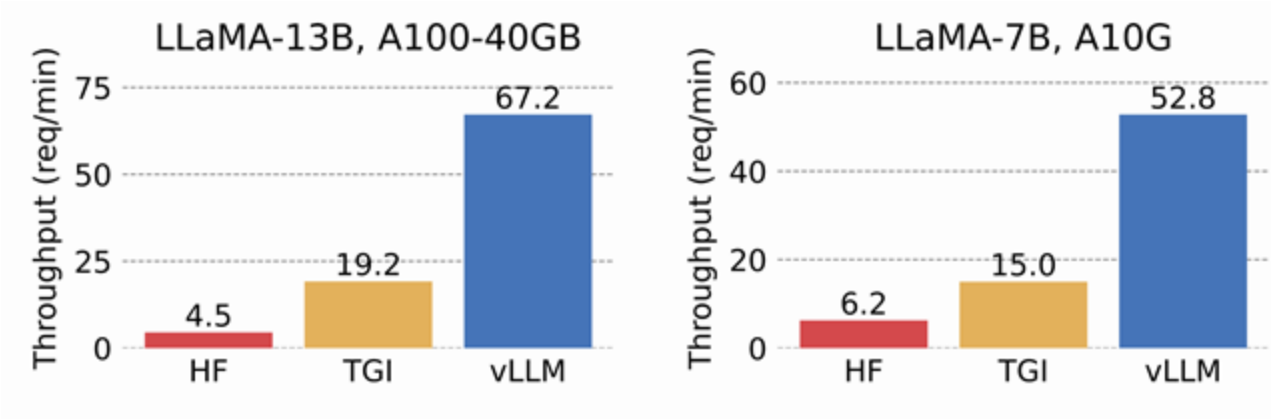


- 각 요청 처리 시, 기존 Huggingface Transformers와 Huggingface Text Generate Inference보다 높은 처리량을 보였음
- A100과 A10 GPU 환경에서 HF보다 **24**배 높은 처리량을 보였으며, TGI보다 **2.5**배 높은 처리량을 보임

https://blog.vllm.ai/2023/06/20/vllm.html

4.1. 성능 비교

Parallel Sampling 작업 시 성능 비교

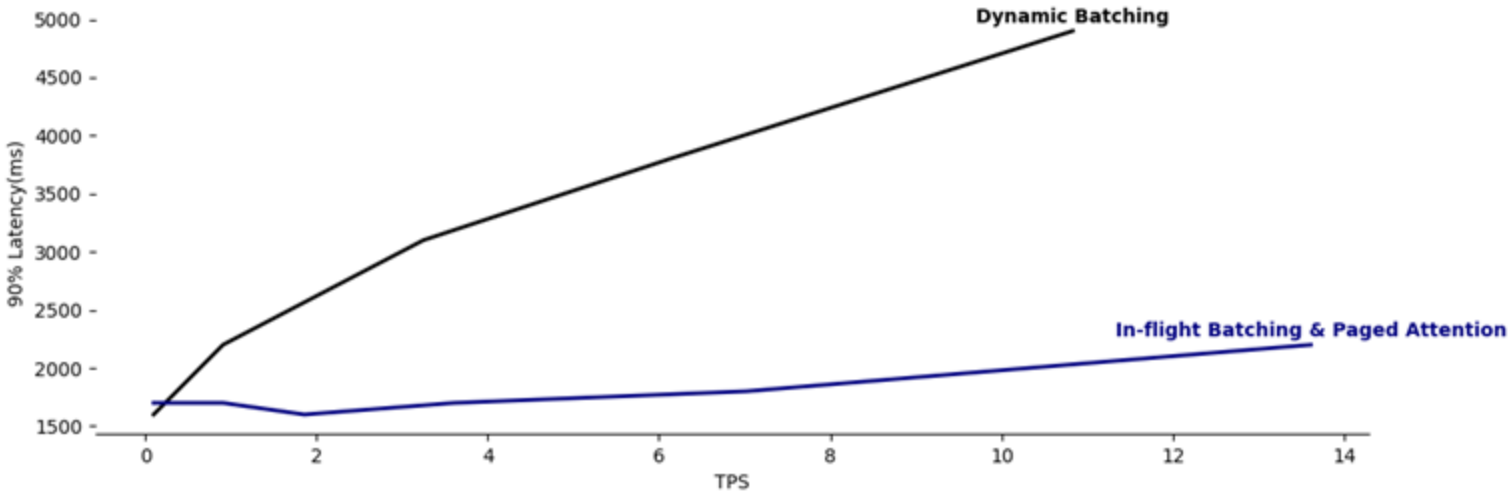


- 3개씩 Parallel Sampling 적용 시, 기존 Huggingface Transformers와 Huggingface Text Generate Inference보다 높은 처리량을 보였음
- A100과 A10 GPU 환경에서 HF보다 **18**배 높은 처리량을 보였으며, TGI보다 **3.5**배 높은 처리량을 보임

https://blog.vllm.ai/2023/06/20/vllm.html

4.1. 성능 비교

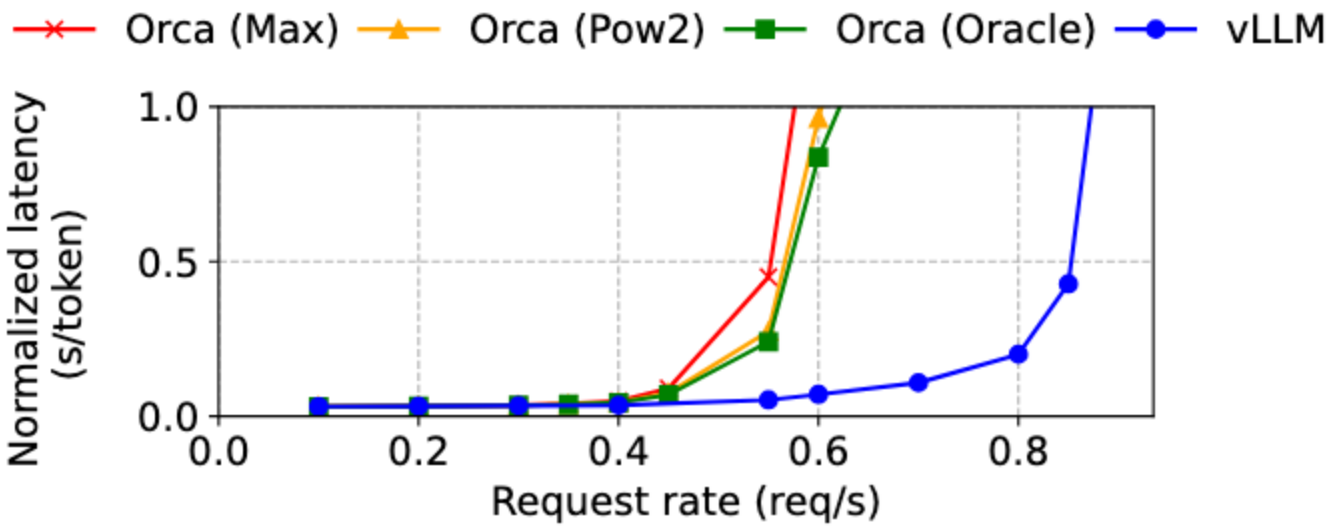
기존 Dynamic Batching과 Continuous Batching(In-flight Batching) 성능 비교



- 비슷한 TPS에서도 연속적으로 배칭 처리를 하는 Continuous Batching이 Dynamic Batching보다 2배 이상 효율적임

4.1. 성능 비교

vLLM의 Continuous Batching 성능의 효율성



- 동일한 Continuous Batching 로직임에도 vLLM은 PagedAttention과 같은 메모리 효율화를 통해 1.5배 이상의 효율성을 보임

4.2. 추가 기능 소개: Speculative Decoding

특정 토큰은 다른 토큰보다 생성하기에 쉽다

- 모든 토큰이 같은 생성 난이도를 가지고 있지 않음

예시: What is the square root of 7? The square root of **7** is **2.646**

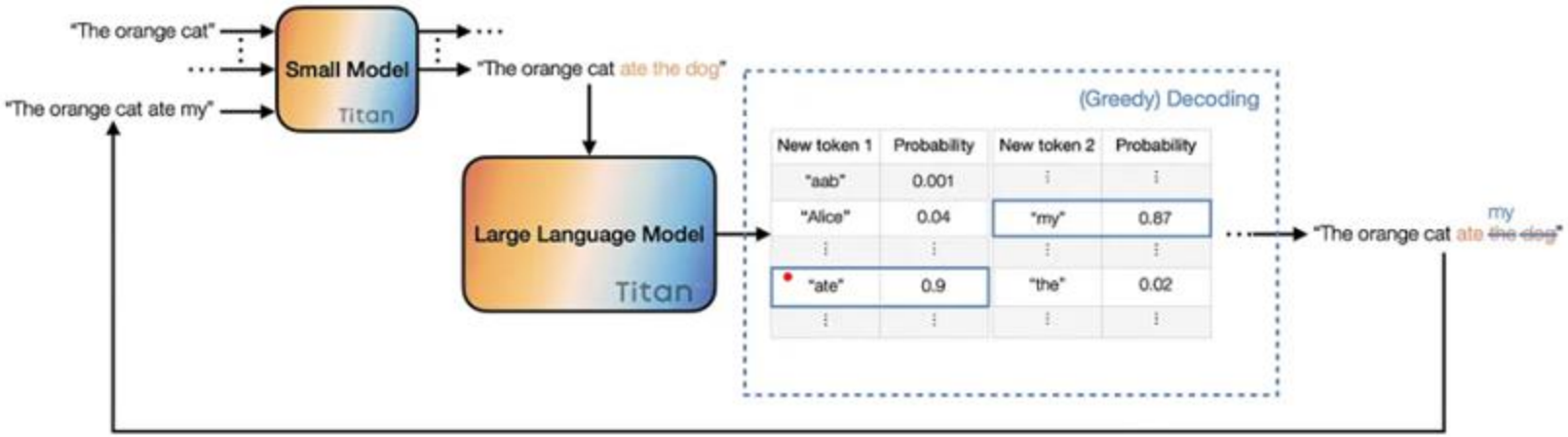
- 위 문장에서 나중에 오는 “7”은 생성하기에 상대적으로 쉬움. 이전에 “square root of”라는 문장이 이미 등장했으므로 이후에 나올 단어를 단순히 복사하는 방식으로 예측할 수 있음.
- 하지만 “2.646”을 생성하는 것은 더 어려움, 모델이 직접 계산하거나 이전에 학습한 답을 기억해야 하기 때문
- 이러한 근거로 작은 모델도 쉬운 답변에 대해서는 사이즈가 더 큰 모델과 유사한 수준의 예측을 할 수 있음

Target Model: 검사를 수행할 main 모델

Draft Model: 임의의 K번을 Decoding 수행할 sub 모델, 일반적으로 Target Model에 비해 훨씬 작은 사이즈이며 Target Model과 같은 구조를 가지는 모델

https://research.google/blog/looking-back-at-speculative-decoding/

4.2. 추가 기능 소개: Speculative Decoding

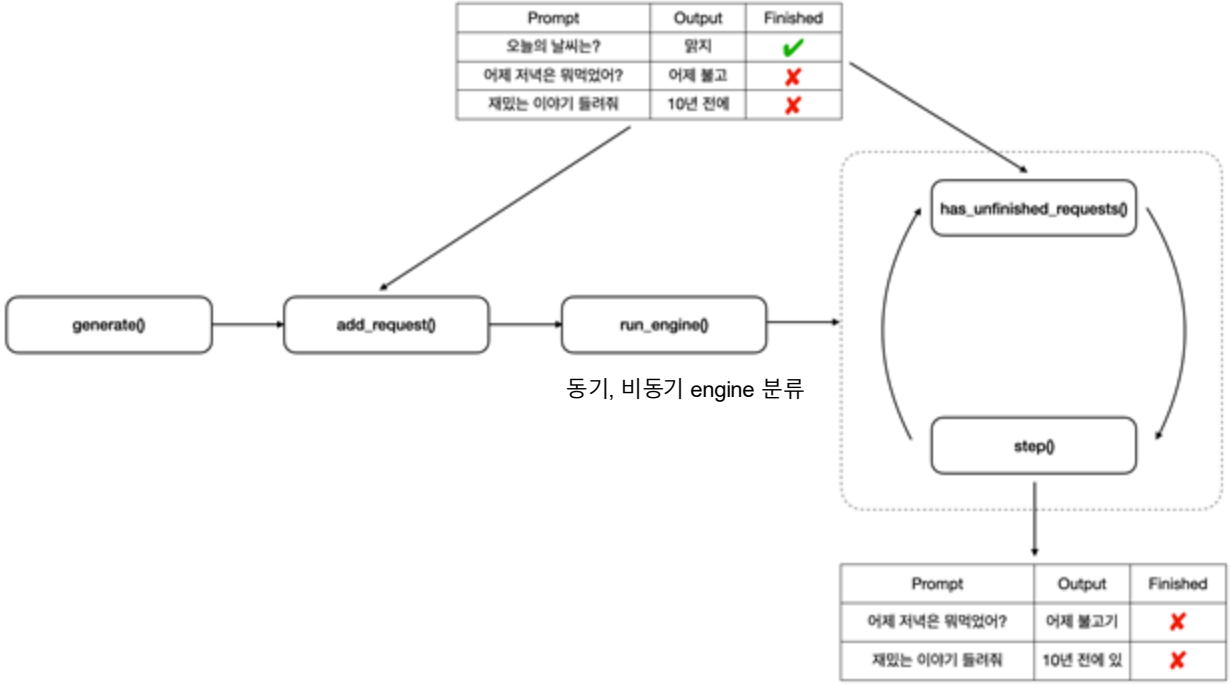


- 임의의 K개 씩 Draft Model은 Decode를 진행 (K번 Decoding)
- Target Model은 K개의 생성된 토큰을 한번에 확률값 검사(K개 당 1번 Decoding) Greedy Decoding 수행 후, 확률값이 다르다면 수정
- 수정 이후 다시 Draft Model은 수정된 토큰 부터 K번 Decoding 수행

https://research.google/blog/looking-back-at-speculative-decoding/

5. 코드 동작 구조 및 실 성능 측정

vLLM 코드 전반적인 코드 동작 구조



동기, 비동기 engine 분류

각 step 마다 하나의 토큰 Decoding

5. 코드 동작 구조 및 실 성능 측정

vLLM 구현 속도를 기존 HF Transformers와 비교

- 실험 환경: Nvidia A100 20GB
- 실험 모델 : 14B 모델의 int4 양자화 버전
- 실험 케이스: 100번의 Request 시 속도 테스트
- 평가 요소: TTFT(Time To First Token), TPS(Token per Second)

Huggingface Transformers

Batch Size	TTFT	TPS
1	0.578s	10.96 token/s
2	1.147s	5.54 token/s
3이상	처리 불가	처리 불가

vLLM

Batch Size	TTFT	TPS
1	0.543s	12.42 token/s
2	1.211s	22.13 token/s
5	2.32s	33.71 token/s

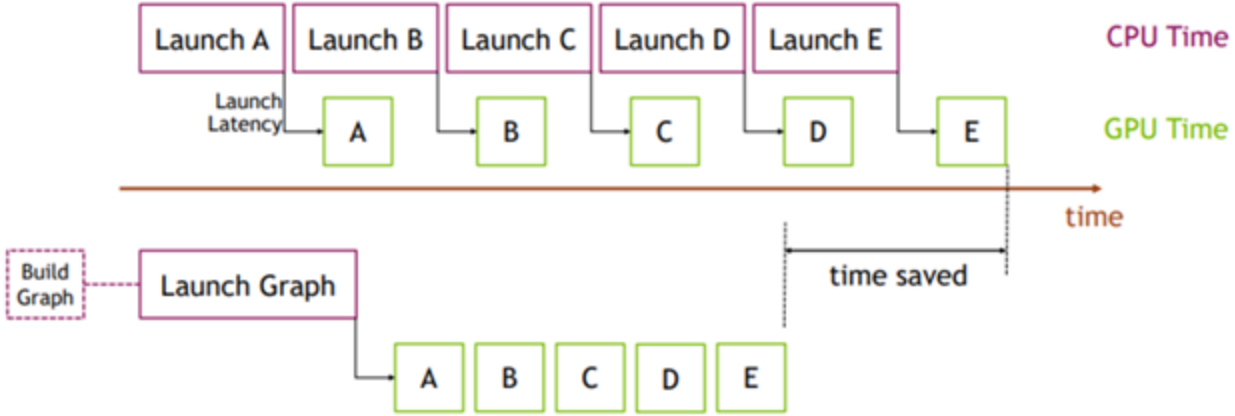
6. 시행 착오

Eager Mode On/Off

- Eager Mode On 시, Cuda Graph Node를 순차적으로 실행함 (OOM 방지, 안정성 증가)
- Eager Mode Off 시, Cuda Graph Capture가 실행되어 미리 연산될 Cuda Graph Node들을 GPU에 저장한 후 실행 (효율성 증가)

Eager Mode On

Eager Mode Off (Cuda Graph Capture)



6. 시행 착오

Eager Mode On/Off

- Eager Mode On시, 양자화 및 Lora Adapter를 추가하면 성능 저하가 되는 이슈가 있었음

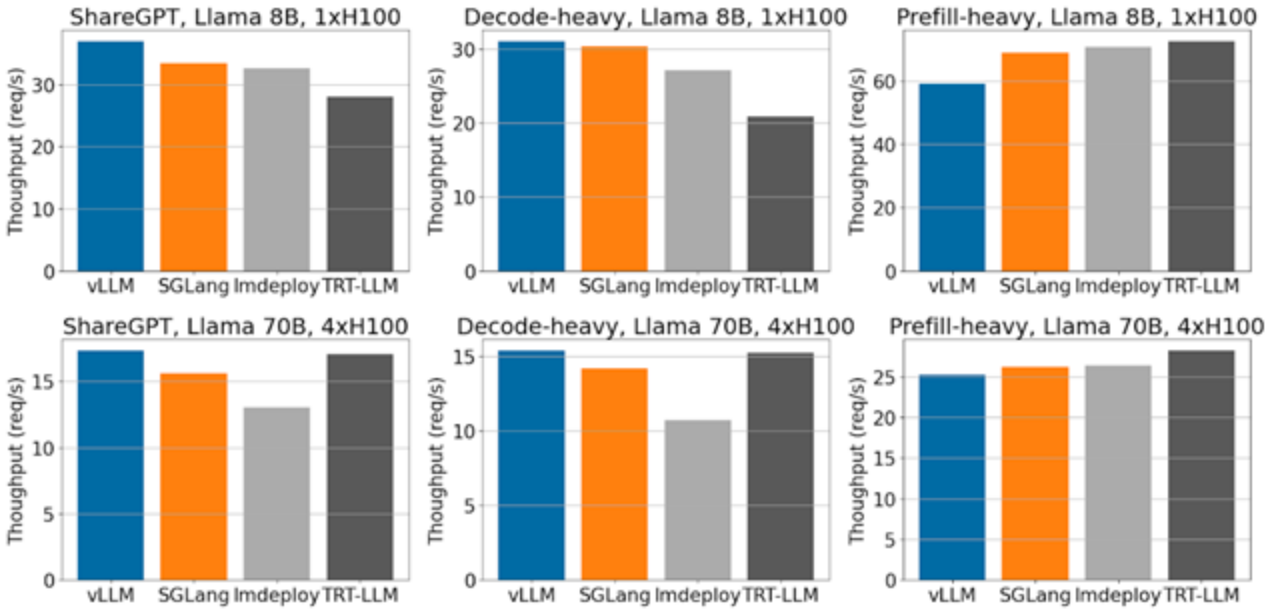
양자화 설정	Throughput (tps)
AWQ (Marlin)	10
Bnb	8
Bnb + LoRA	5
AWQ + LoRA	4

- Eager Mode Off시, 성능 저하 이슈가 해결되었음

7. LLM 프레임워크 비교

- vLLM
  - 다양한 모델 호환성 및 대규모 트래픽에 일관된 성능을 보장하여 안정성이 높은 프레임워크
  - real-stream 애플리케이션에 적합 (Continuous Batching 지원)
- TensorRT-LLM
  - Nvidia GPU에 최적화된 TensorRT Engine 기반 추론 성능 최적화 프레임워크
  - 학습된 모델을 onnx로 변환 후 tensor engine 파일로 변환해야하는 추가 작업 필요, 변환이 지원되지 않는 모델이 있음
  - Nvidia GPU외의 다른 하드웨어는 호환되지 않음
- LMDeploy
  - 낮은 리소스 사용으로 최적 추론 속도를 제공하는 프레임워크
  - edge computing과 같은 작은 weight 모델에 적합함
- SgLang
  - RadixAttention이라는 KV Cache 재활용 방법론을 통해 실시간 추론 스트리밍 최적화
  - vLLM과 마찬가지로 다양한 모델을 호환하며 real-stream 애플리케이션에 적합
  - 빠르게 발전 중인 프레임워크이나 관련 문서 및 정보 미흡

7. LLM 프레임워크 비교



- vLLM은 일반적으로 가장 높은 성능을 보임, 다른 프레임워크 들에 비해 TTFT 처리량이 느려, Prefill-heavy 작업에는 낮은 성능을 보였음
- Prefill-heavy 작업의 경우 Tensor-LLM이 가장 높은 처리량을 보였으나 GPU가 적을 경우 일반적인 Decode 작업이 느림
- SGLang은 일반적으로 vLLM보다 조금 처리량이 낮았으나, Prefill-heavy작업에서는 더욱더 높은 처리량을 보였음

https://www.hyperstack.cloud/blog/case-study/what-is-vllm-a-guide-to-quick-inference

출처

- https://tech.scatterlab.co.kr/vllm-implementation-details/
- https://arxiv.org/pdf/2309.06180
- https://docs.vllm.ai/en/stable/design/kernel/paged_attention.html
- https://blog.vllm.ai/2023/06/20/vllm.html
- https://pytorch.org/blog/accelerating-pytorch-with-cuda-graphs/
- https://velog.io/@hanhan/vLLM